Code Alpha

Machine Learning

TASK 1: Credit Scoring Model

Objective: Predict an individual's creditworthiness using past financial data.

Approach: Use classification algorithms like Logistic Regression, Decision Trees, or Random Forest.

Key Features:

- Feature engineering from financial history.
- Model accuracy assessment using metrics like Precision, Recall, F1-Score, ROC-AUC.
- Dataset could include: income, debts, payment history, etc.

Import Libraries

We start by importing the key Python libraries for data handling and analysis:

- pandas helps us load, explore, and clean the dataset efficiently.
- numpy allows numerical computations like ratios, percentages, or handling missing values.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("/content/Credit_scoring.csv")
```

```
/tmp/ipython-input-458814466.py:6: DtypeWarning: Columns (26) have mixed types. Specify dtype
  df = pd.read_csv("/content/Credit_scoring.csv")
```

head()→ Shows the first 5 rows to understand how data looks.

```
df.head()
```

	ID	Customer_ID	Month	Name	Age	SSN	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	...	Credit_Mix	Outstanding_Debt	Cre
0	0x1602	CUS_0xd40	January	Aaron Maashoh	23	821-00-0265	Scientist	19114.12	1824.843333	3	...	_	809.98	
1	0x1603	CUS_0xd40	February	Aaron Maashoh	23	821-00-0265	Scientist	19114.12	NaN	3	...	Good	809.98	
2	0x1604	CUS_0xd40	March	Aaron Maashoh	-500	821-00-0265	Scientist	19114.12	NaN	3	...	Good	809.98	
3	0x1605	CUS_0xd40	April	Aaron Maashoh	23	821-00-0265	Scientist	19114.12	NaN	3	...	Good	809.98	
4	0x1606	CUS_0xd40	May	Aaron Maashoh	23	821-00-0265	Scientist	19114.12	1824.843333	3	...	Good	809.98	

5 rows × 28 columns

info()→ Displays column names, data types, and non-null counts.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 28 columns):
 #   Column                    Non-Null Count   Dtype
---  ------                    --------------   -----
 0   ID                        100000 non-null  object
 1   Customer_ID               100000 non-null  object
 2   Month                     100000 non-null  object
 3   Name                      90015 non-null   object
 4   Age                       100000 non-null  object
 5   SSN                       100000 non-null  object
 6   Occupation                100000 non-null  object
 7   Annual_Income             100000 non-null  object
 8   Monthly_Inhand_Salary     84998 non-null   float64
 9   Num_Bank_Accounts         100000 non-null  int64
 10  Num_Credit_Card           100000 non-null  int64
 11  Interest_Rate             100000 non-null  int64
 12  Num_of_Loan               100000 non-null  object
 13  Type_of_Loan              88592 non-null   object
 14  Delay_from_due_date       100000 non-null  int64
 15  Num_of_Delayed_Payment    92998 non-null   object
 16  Changed_Credit_Limit      100000 non-null  object
 17  Num_Credit_Inquiries      98035 non-null   float64
 18  Credit_Mix                100000 non-null  object
 19  Outstanding_Debt          100000 non-null  object
 20  Credit_Utilization_Ratio  100000 non-null  float64
 21  Credit_History_Age        90970 non-null   object
 22  Payment_of_Min_Amount     100000 non-null  object
 23  Total_EMI_per_month       100000 non-null  float64
 24  Amount_invested_monthly   95521 non-null   object
 25  Payment_Behaviour         100000 non-null  object
 26  Monthly_Balance           98800 non-null   object
 27  Credit_Score              100000 non-null  object
dtypes: float64(4), int64(4), object(20)
memory usage: 21.4+ MB
```

shape→ Gives the number of rows and columns.

```
df.shape
```

```
(100000, 28)
```

Columns

Lists all column names to know which features are available.

duplicated().sum()

Counts duplicate rows, if any.

```
df.columns
```

```
Index(['ID', 'Customer_ID', 'Month', 'Name', 'Age', 'SSN', 'Occupation',
       'Annual_Income', 'Monthly_Inhand_Salary', 'Num_Bank_Accounts',
       'Num_Credit_Card', 'Interest_Rate', 'Num_of_Loan', 'Type_of_Loan',
       'Delay_from_due_date', 'Num_of_Delayed_Payment', 'Changed_Credit_Limit',
       'Num_Credit_Inquiries', 'Credit_Mix', 'Outstanding_Debt',
       'Credit_Utilization_Ratio', 'Credit_History_Age',
       'Payment_of_Min_Amount', 'Total_EMI_per_month',
       'Amount_invested_monthly', 'Payment_Behaviour', 'Monthly_Balance',
       'Credit_Score'],
      dtype='object')
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

describe()→ Provides summary stats (mean, min, max, std) for numerical columns.

```
df.describe()
```

	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card	Interest_Rate	Delay_from_due_date	Num_Credit_Inquiries	Credit_Utilization_Ratio	Total_EMI_per_month
count	84998.000000	100000.000000	100000.00000	100000.000000	100000.000000	98035.000000	100000.000000	100000.000000
mean	4194.170850	17.091280	22.47443	72.466040	21.068780	27.754251	32.285173	1403.118217
std	3183.686167	117.404834	129.05741	466.422621	14.860104	193.177339	5.116875	8306.041270
min	303.645417	-1.000000	0.00000	1.000000	-5.000000	0.000000	20.000000	0.000000
25%	1625.568229	3.000000	4.00000	8.000000	10.000000	3.000000	28.052567	30.306660
50%	3093.745000	6.000000	5.00000	13.000000	18.000000	6.000000	32.305784	69.249473
75%	5957.448333	7.000000	7.00000	20.000000	28.000000	9.000000	36.496663	161.224249
max	15204.633333	1798.000000	1499.00000	5797.000000	67.000000	2597.000000	50.000000	82331.000000

Handle Missing Values

Missing values can cause errors or reduce model accuracy. We drop rows with missing values for simplicity, though imputation is another option. This ensures all data used for training is clean and complete.

```
df = df.dropna()
df['Credit_Score'].value_counts()
```

Credit_Score	count
2	44373
1	24165
0	14778

dtype: int64

Encode Categorical Variables

Machine learning models require numerical inputs, so we encode categorical columns. Label Encoder converts string categories into numbers for easier processing. This allows models like Logistic Regression or Random Forest to understand these features.

Feature Engineering

We create new features from existing data to improve model performance. Examples include Debt-to-Income ratio or Credit Utilization, which are predictive of creditworthiness. Feature engineering helps the model capture important financial patterns.

```
#For Encoding Variables
from sklearn.preprocessing import LabelEncoder
categorical_cols = df.select_dtypes(include = ['object']).columns
for col in categorical_cols:
    le = LabelEncoder()

    df[col] = df[col].astype(str)
    df[col] = le.fit_transform(df[col])
```

```
#For Feature Engineering
if 'income' in df.columns and 'debt' in df.columns:
    df["debt_to_income"] = df["debt"] / df["income"]

if 'current_balance' in df.columns and 'credit_limit' in df.columns:
    df['Credit_Utilization'] = df['current_balance']/df['credit_limit']
```

Split Dataset

We separate features (X) and target (y) for supervised learning. Then we split into training (80%) and testing (20%) sets to evaluate the model on unseen data. This ensures fair assessment and avoids overfitting.

Scale Features

Scaling standardizes feature values so models treat them equally. Standard Scaler adjusts numerical features to zero mean and unit variance. Scaling is especially important for algorithms like Logistic Regression.

```
#Assuming 'Credit_Score' is the target, convert to binary

df['Target'] = df['Credit_Score'].apply(lambda x:1 if x == 2 else 0)
X = df.drop(['Credit_Score','Target'],axis = 1)
y = df['Target']
```

```
#Train-test split
from sklearn.model_selection import train_test_split, GridSearchCV

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size= 0.2, random_state=42)
```

```
from sklearn.preprocessing import StandardScaler

#scale features for Logistic Regression
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Train Models

We train three models: Logistic Regression, Decision Tree, and Random Forest. Each model learns patterns from the training data to predict creditworthiness. This gives us multiple options to compare and select the best-performing model.

```
#Logistic Regression
from sklearn.linear_model import LogisticRegression
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
X_train_scaled_imputed = imputer.fit_transform(X_train_scaled)
X_test_scaled_imputed = imputer.transform(X_test_scaled)

lr = LogisticRegression()
lr.fit(X_train_scaled_imputed, y_train)
y_pred_lr = lr.predict(X_test_scaled_imputed)
```

```
#Decision Tree
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train,y_train)
y_pred_dt = dt.predict(X_test)
```

```
#Random Forest
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
```

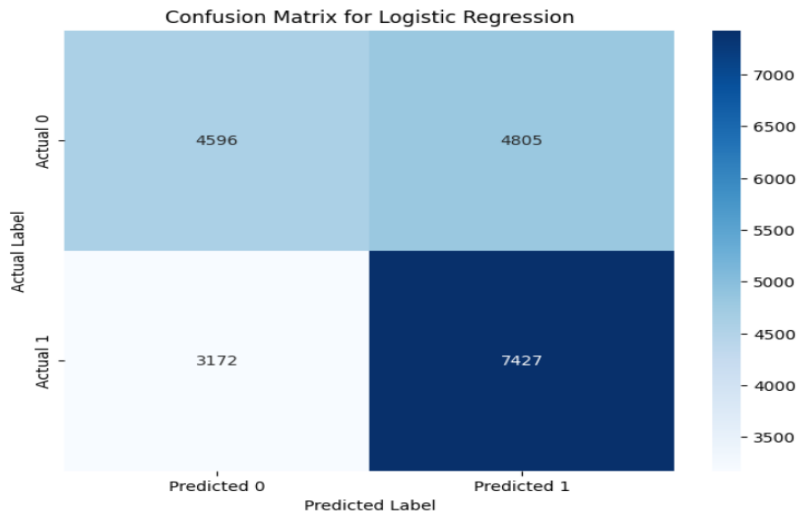
Evaluate Models

We measure model performance using metrics: accuracy, precision, recall, F1-score, and ROC AUC. Confusion matrix helps identify correct and incorrect predictions for each class. Evaluation ensures we understand how well our models generalize to new data.

```
#Evalute Models
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score
from sklearn.metrics import confusion_matrix

print("Logistic Regression Report: \n", classification_report(y_test, y_pred_lr))
# Get the confusion matrix for Logistic Regression
cm_lr = confusion_matrix(y_test, y_pred_lr)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix for Logistic Regression')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()

print("Decision Tree Report: \n", classification_report(y_test, y_pred_dt))
# Get the confusion matrix for Decision Tree
cm_dt = confusion_matrix(y_test, y_pred_dt)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='YlOrRd',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix for Decision Tree')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()

print("Random Forest Report: \n", classification_report(y_test, y_pred_rf))
# Get the confusion matrix for Random Forest
cm_rf = confusion_matrix(y_test, y_pred_rf)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='magma',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.title('Confusion Matrix for Random Forest')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()
```

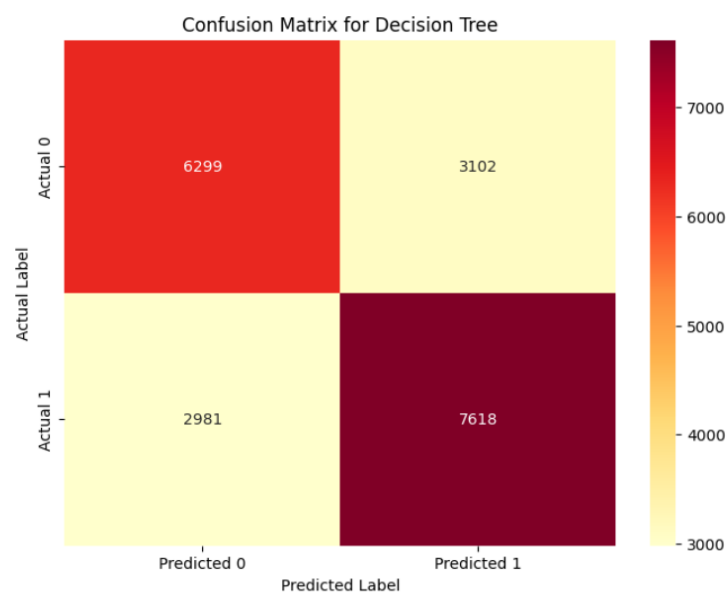
Confusion Matrix for Logistic Regression

```
Logistic Regression Report:
              precision    recall  f1-score   support

           0       0.59      0.49      0.54      9401
           1       0.61      0.70      0.65     10599

    accuracy                           0.60     20000
   macro avg       0.60      0.59      0.59     20000
weighted avg       0.60      0.60      0.60     20000
```



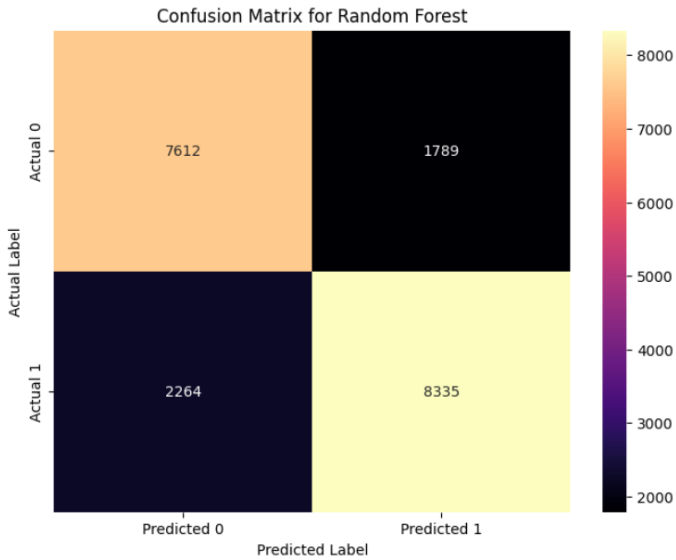
Confusion Matrix for Decision Tree

```
Decision Tree Report:
              precision    recall  f1-score   support

           0       0.68      0.67      0.67      9401
           1       0.71      0.72      0.71     10599

    accuracy                           0.70     20000
   macro avg       0.69      0.69      0.69     20000
weighted avg       0.70      0.70      0.70     20000
```



Confusion Matrix for Random Forest

```
Random Forest Report:
              precision    recall  f1-score   support

           0       0.77      0.81      0.79      9401
           1       0.82      0.79      0.80     10599

    accuracy                           0.80     20000
   macro avg       0.80      0.80      0.80     20000
weighted avg       0.80      0.80      0.80     20000
```



Save the Best Model

We save the trained model using joblib for future use. This allows us to load the model anytime without retraining, enabling real-time predictions. Saving is crucial for deploying the model in apps or production systems.

```
#ROC-AUC Scores
print("Logistic Regression ROC-AUC: ",roc_auc_score(y_test, lr.predict_proba(X_test_scaled_imputed)[:,1]))
print("Decision Tree ROC-AUC: ",roc_auc_score(y_test, dt.predict_proba(X_test)[:,1]))
print("Random Forest ROC-AUC: ",roc_auc_score(y_test, rf.predict_proba(X_test)[:,1]))
```

```
Logistic Regression ROC-AUC:  0.6295272099244811
Decision Tree ROC-AUC:  0.6943910771286483
Random Forest ROC-AUC:  0.8601540914817776
```

```
#Save the best Model
import joblib
joblib.dump(rf, 'random_forest_model.pkl')
```

```
['random_forest_model.pkl']
```